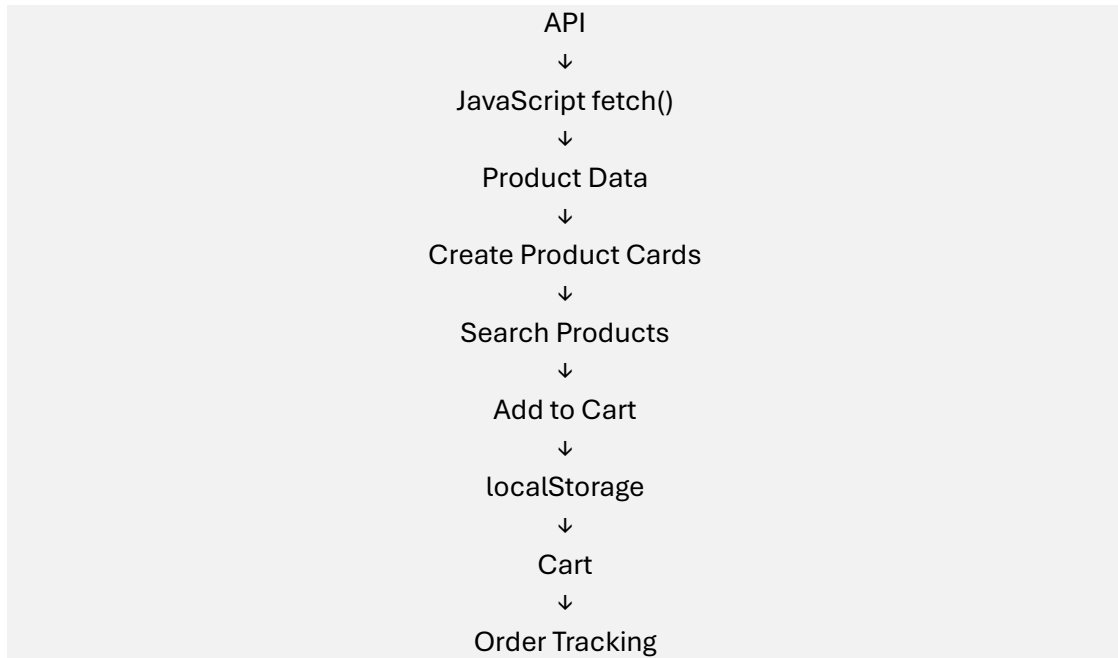


E-Commerce application (HTML, CSS, JavaScript)

"In a real e-commerce website, products are not written manually in HTML. JavaScript gets product data from a server/API and creates the product cards dynamically."

simple flow:



Use **Fake Store API** for the hands-on:

<https://fakestoreapi.com/products>

we don't need a backend for this experiment.

2. Project Setup

Create:

```
ecommerce-js/  
|  
├── index.html  
├── style.css  
└── script.js
```

3. HTML

create index.html.

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>KL E-Commerce</title>  
  <link rel="stylesheet" href="style.css">  
</head>
```

```

<body>

  <h1>&img alt="shopping cart icon" data-bbox="238 141 258 154"/> KL E-Commerce Store</h1>

  <!-- Search -->
  <input
    type="text"
    id="searchBox"
    placeholder="Search products..."
  >

  <!-- Products -->
  <h2>Products</h2>
  <div id="productContainer"></div>

  <!-- Cart -->
  <h2>Shopping Cart</h2>
  <div id="cartContainer"></div>

  <h3>
    Cart Total: ₹<span id="cartTotal">0</span>
  </h3>

  <button onclick="placeOrder()">Place Order</button>

  <!-- Order Tracking -->
  <h2>Order Tracking</h2>
  <div id="orderContainer"></div>

  <script src="script.js"></script>

</body>
</html>

```

Important concepts:

id="productContainer"

JavaScript will find this element and insert products.

id="searchBox"

JavaScript will read what the user types.

onclick="placeOrder()"

Calls a JavaScript function.

4. Basic CSS

simple styling.

```
body {  
  font-family: Arial;  
  margin: 30px;  
}  
  
#searchBox {  
  width: 300px;  
  padding: 10px;  
  margin-bottom: 20px;  
}  
  
#productContainer {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  gap: 20px;  
}  
  
.product {  
  border: 1px solid #ccc;  
  padding: 15px;  
  border-radius: 10px;  
}  
  
.product img {  
  width: 100px;  
  height: 120px;  
  object-fit: contain;  
}  
  
button {  
  padding: 8px 15px;  
  margin: 5px;  
  cursor: pointer;  
}
```

5. API-Based Product Loading

Step 1 — Create variables

```
let products = [];  
  
let cart = JSON.parse(localStorage.getItem("cart")) || [];
```

Description:

```
let products = [];
```

Stores products received from API.

localStorage

Stores data in the browser.

JSON.parse()

Converts JSON text back into JavaScript data.

Step 2 — Fetch products

```
async function loadProducts() {  
  
  const response = await fetch(  
    "https://fakestoreapi.com/products"  
  );  
  
  products = await response.json();  
  
  displayProducts(products);  
}  
  
loadProducts();
```

Explain this very simply:

```
fetch()  
↓  
Request API  
↓  
response  
↓  
response.json()  
↓  
JavaScript Object  
↓  
displayProducts()
```

"fetch()" is used to communicate with an API."

6. Dynamic Product Catalog

dynamically create product cards.

```
function displayProducts(productList) {  
  
  const container =  
    document.getElementById("productContainer");
```

```

container.innerHTML = "";

productList.forEach(product => {

  container.innerHTML += `
    <div class="product">

      <h3>${product.title}</h3>

      <p>₹${product.price}</p>

      <button onclick="addToCart(${product.id})">
        Add to Cart
      </button>

    </div>
  `;
});
}

```

Description:

productList.forEach(product => {

Means:

"Take each product one by one."

Then:

`${product.title}`

gets the product name.

`${product.price}`

gets the price.

`${product.image}`

gets the image.

7. Product Search

```

document.getElementById("searchBox")
  .addEventListener("input", function() {

    const keyword = this.value.toLowerCase();

    const filteredProducts = products.filter(product =>

```

```
    product.title.toLowerCase().includes(keyword)
  );

  displayProducts(filteredProducts);
});
```

Description:

```
User types "shirt"
  ↓
input event
  ↓
filter products
  ↓
display matching products
```

The important JavaScript concepts covered here are:

- addEventListener()
- filter()
- includes()
- toLowerCase()

8. Shopping Cart

Now implement the shopping cart.

Add product

```
function addToCart(productId) {

  const product = products.find(
    p => p.id === productId
  );

  cart.push(product);

  localStorage.setItem(
    "cart",
    JSON.stringify(cart)
  );

  displayCart();
}
```

Description:

find()

Finds one product.

push()

Adds product to cart.

JSON.stringify()

Converts JavaScript data into text so it can be stored.

Display cart

```
function displayCart() {  
  
  const container =  
    document.getElementById("cartContainer");  
  
  container.innerHTML = "";  
  
  let total = 0;  
  
  cart.forEach((product, index) => {  
  
    total += product.price;  
  
    container.innerHTML += `  
      <p>  
        ${product.title}  
        - ₹${product.price}  
  
        <button onclick="removeFromCart(${index})">  
          Remove  
        </button>  
      </p>  
    `;  
  });  
  
  document.getElementById("cartTotal")  
    .innerText = total.toFixed(2);  
}
```

Remove from cart

```
function removeFromCart(index) {  
  
  cart.splice(index, 1);  
  
  localStorage.setItem(  
    "cart",  
    JSON.stringify(cart)
```

```
);  
  
displayCart();  
}  
  
displayCart();
```

9. Order Tracking Dashboard

the same JavaScript concepts can be used to create a simple dashboard.

```
function placeOrder() {  
  
    if (cart.length === 0) {  
        alert("Cart is empty!");  
        return;  
    }  
  
    document.getElementById("orderContainer").innerHTML = `  
        <div class="product">  
  
            <h3>Order #1001</h3>  
  
            <p>Order Placed..</p>  
            <p>Packed..</p>  
            <p>Shipped..</p>  
            <p>Out for Delivery..</p>  
  
        </div>  
    `;  
  
    cart = [];  
  
    localStorage.removeItem("cart");  
  
    displayCart();  
  
    alert("Order placed successfully!");  
}
```

10. Final script.js

the complete JavaScript file.

```
let products = [];  
  
let cart = JSON.parse(localStorage.getItem("cart")) || [];
```



```
// =====
// 1. LOAD PRODUCTS FROM API
// =====

async function loadProducts() {

  const response = await fetch(
    "https://fakestoreapi.com/products"
  );

  products = await response.json();

  displayProducts(products);
}

// =====
// 2. DISPLAY PRODUCTS
// =====

function displayProducts(productList) {

  const container =
    document.getElementById("productContainer");

  container.innerHTML = "";

  productList.forEach(product => {

    container.innerHTML += `
      <div class="product">

        <h3>${product.title}</h3>

        <p>₹${product.price}</p>

        <button onclick="addToCart(${product.id})">
          Add to Cart
        </button>

      </div>
    `;
  });
}
```

```

// =====
// 3. SEARCH PRODUCTS
// =====

document.getElementById("searchBox")
  .addEventListener("input", function() {

    const keyword = this.value.toLowerCase();

    const filteredProducts = products.filter(product =>
      product.title.toLowerCase().includes(keyword)
    );

    displayProducts(filteredProducts);
  });

// =====
// 4. ADD TO CART
// =====

function addToCart(productId) {

  const product = products.find(
    p => p.id === productId
  );

  cart.push(product);

  localStorage.setItem(
    "cart",
    JSON.stringify(cart)
  );

  displayCart();
}

// =====
// 5. DISPLAY CART
// =====

function displayCart() {

  const container =
    document.getElementById("cartContainer");

```

```

container.innerHTML = "";

let total = 0;

cart.forEach((product, index) => {

    total += product.price;

    container.innerHTML += `
        <p>
            ${product.title}
            - ₹${product.price}

            <button onclick="removeFromCart(${index})">
                Remove
            </button>
        </p>
    `;
});

document.getElementById("cartTotal")
    .innerText = total.toFixed(2);
}

// =====
// 6. REMOVE FROM CART
// =====

function removeFromCart(index) {

    cart.splice(index, 1);

    localStorage.setItem(
        "cart",
        JSON.stringify(cart)
    );

    displayCart();
}

// =====
// 7. PLACE ORDER
// =====

function placeOrder() {

```

```

if (cart.length === 0) {

    alert("Cart is empty!");

    return;
}

document.getElementById("orderContainer").innerHTML = `
    <div class="product">

        <h3>Order #1001</h3>

        <p>Order Placed..</p>
        <p>Packed...</p>
        <p>Shipped..</p>
        <p>Out for Delivery..</p>

    </div>
`;

cart = [];

localStorage.removeItem("cart");

displayCart();

alert("Order placed successfully!");
}

// =====
// 8. INITIALIZE APPLICATION
// =====

loadProducts();

displayCart();

```

The 8 JavaScript Concepts Students Must Remember

1. fetch() → Get products from API
2. async/await → Wait for API response
3. forEach() → Display every product
4. filter() → Search products

5. find() → Find selected product

6. push() → Add product to cart

7. localStorage → Save cart in browser

8. JSON → Convert/store data